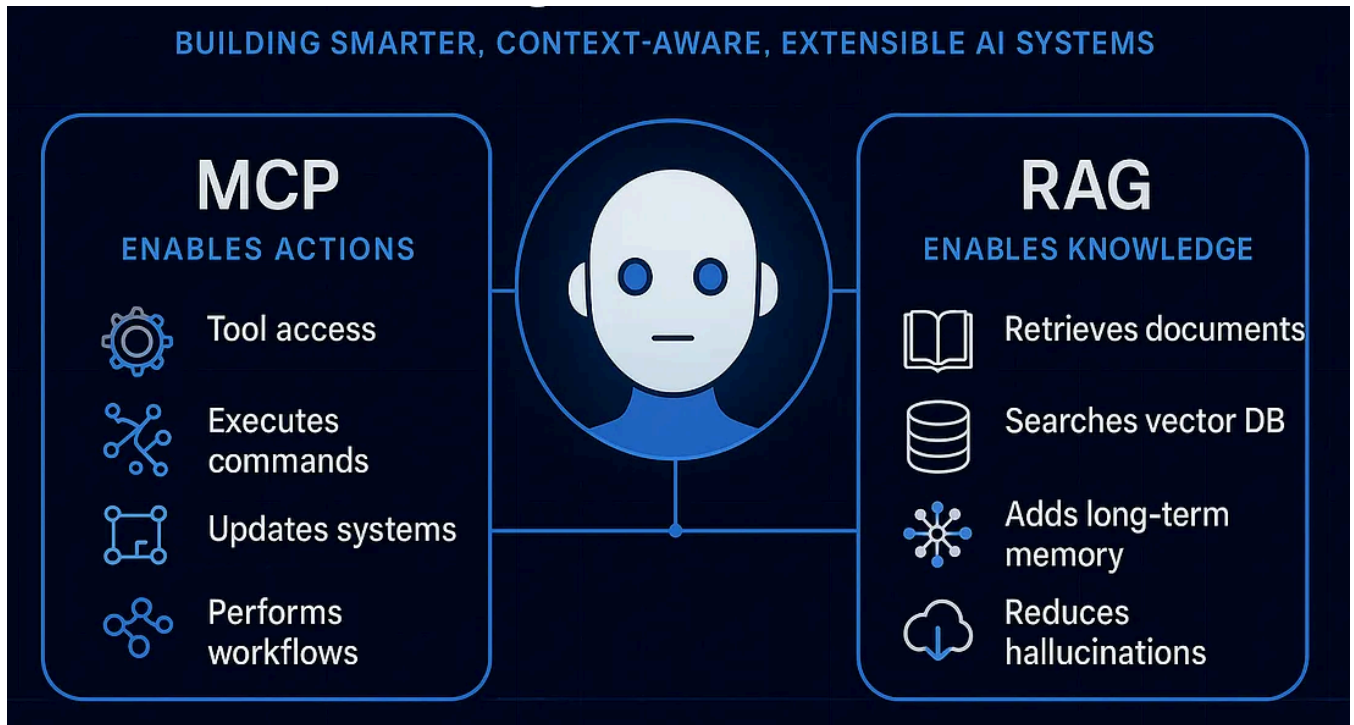


★ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



Dev Genius · [Follow publication](#)



AI Agents powered through MCP & RAG

AI Agents powered through MCP & RAG

Building Smarter, Context-Aware, Extensible AI Systems

3 min read · Nov 29, 2025



Vishal Dhok

Follow



Listen



Share

... More

Artificial intelligence agents are rapidly moving from simple chat interfaces to fully autonomous systems that can reason, take actions, and integrate deeply with enterprise environments. Two technologies Model Context Protocol (MCP) and

Retrieval-Augmented Generation (RAG) are emerging as foundational building blocks for these next-generation agents.

In this article, we'll break down what MCP and RAG actually are, how they differ, and how combining them creates powerful, context-rich, and action oriented AI behavior.

Introduction to MCP & RAG

What is MCP (Model Context Protocol)?

MCP is an open protocol that enables AI agents to securely interact with external tools, APIs, databases, and systems. Think of it as an “operating system layer” for AI — a standardized way for models to perform actions, fetch data, or control software.

MCP gives agents:

- Tool access (APIs, databases, microservices)
- Real-time data retrieval
- Safe, permission-controlled operations
- Extensibility without retraining the model

It separates *capabilities* from the model itself, making agents more modular and enterprise ready.

What is RAG (Retrieval-Augmented Generation)?

RAG enhances LLMs by giving them access to knowledge bases via embeddings and vector search. Instead of relying purely on the model's internal memory, RAG provides accurate, up to date, domain specific information on demand.

RAG gives agents:

- Accurate factual responses
- Long-term memory via documents & embeddings
- Personalized or enterprise-specific knowledge retrieval
- Reduced hallucinations

In short: RAG allows the model to *remember and reason* using external knowledge.

Aspect	MCP	RAG
Purpose	Enables actions, tooling, and system integration	Enables knowledge retrieval & context enrichment
Focus Area	Doing (operations, commands, workflows)	Knowing (facts, documents, history, memory)
Data Source	APIs, tools, live systems	Vector databases, document stores
Output	System actions, structured data	Context paragraphs, retrieved knowledge
Primary Benefit	Extends agent capabilities	Improves reasoning accuracy & memory

Together, they complement each other: **MCP lets the agent act; RAG lets the agent think.**

MCP Vs RAG

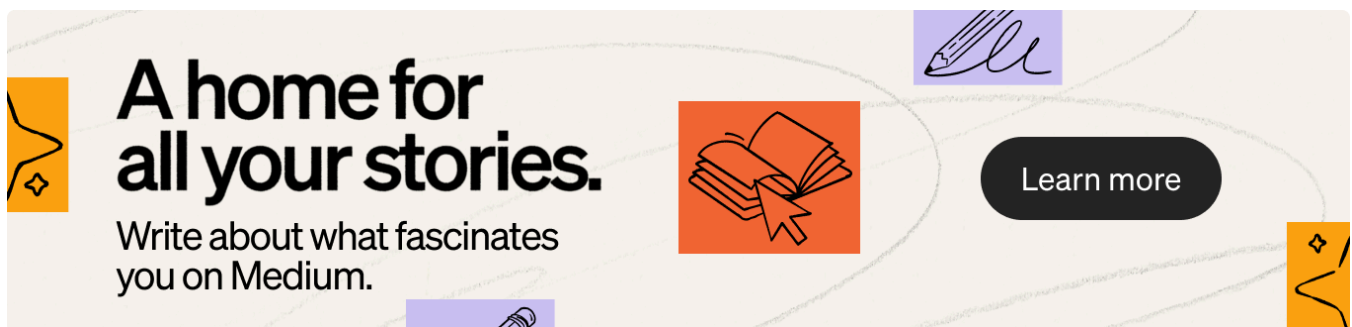
How they can power AI Agents

1. Build Context-Aware Autonomous Agents

RAG provides agents with background knowledge, past decisions, project history, and customer context. MCP lets the agent use that context to perform actions — creating tickets, updating systems, running queries, etc.

2. Enable Real-Time Decision Making

RAG fetches relevant data. MCP executes decisions based on that data.



Example:

- RAG retrieves “top customer issues from the last quarter.”
- MCP creates JIRA tickets, updates dashboards, or triggers workflows.

3. Reduce Hallucinations & Increase Safety

RAG grounds the model in verifiable knowledge. MCP ensures actions go through controlled tools with permissions.

4. Power Multi-Step Workflows & Agents

RAG helps the agent understand tasks and prior state. MCP lets the agent break down tasks and interact with 3rd-party systems like GitHub, Snowflake, Slack, Salesforce, or internal APIs.

Real-World Example: An AI Engineering Assistant

Here's how RAG + MCP power a full engineering-grade AI agent.

Scenario: A Coding Agent Helping a Software Team

Step 1 — Developer asks: “Help me debug why the payment API is failing.”

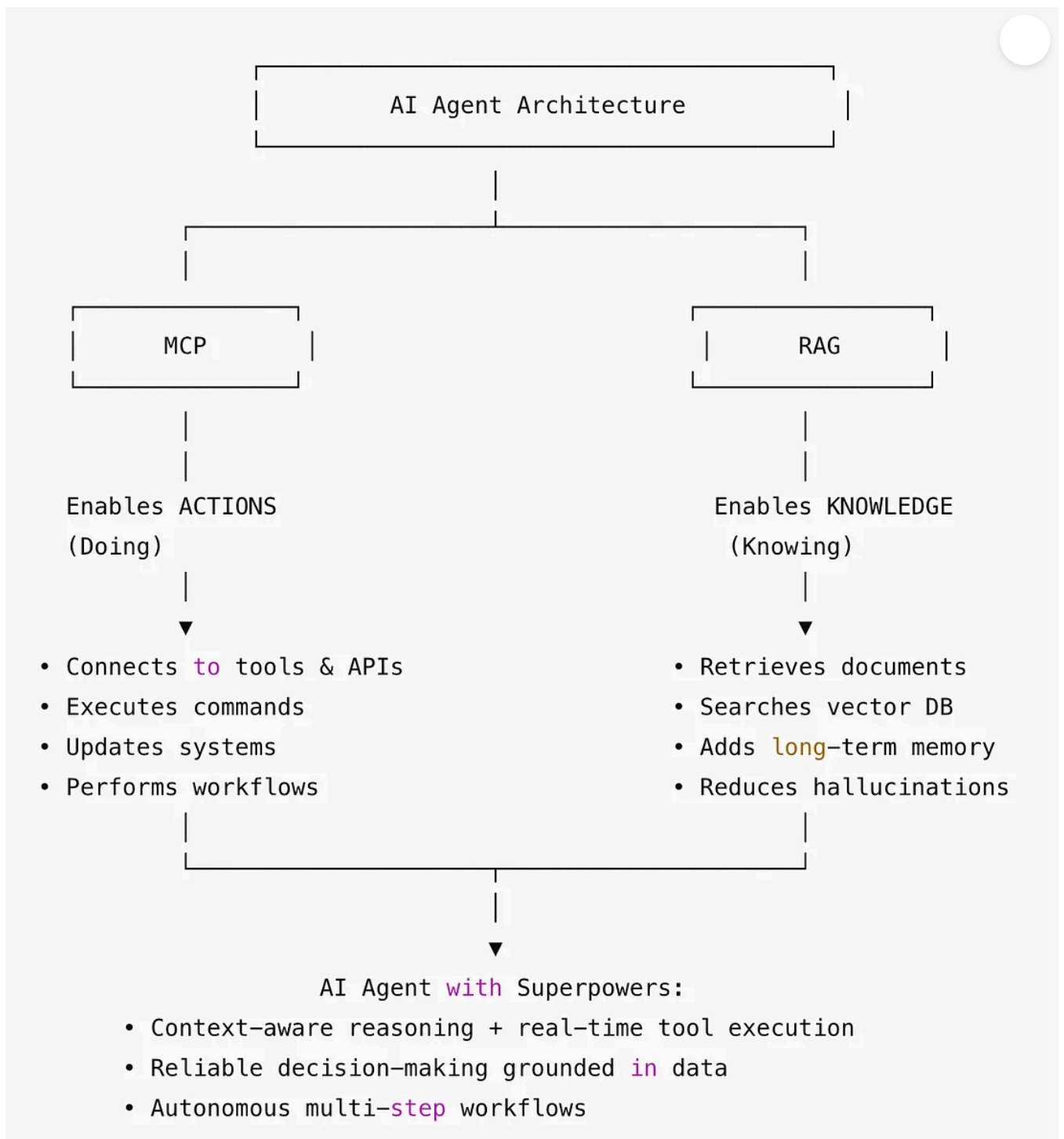
Using RAG:

- Agent retrieves documentation on payment API
- Fetches error logs from prior incidents
- Retrieves recent commits related to payment service

Using MCP:

- Runs diagnostic checks on the API
- Queries the monitoring system (Datadog/Splunk)
- Suggests a code fix and raises a pull request
- Notifies the developer in Slack

Outcome: An end-to-end autonomous engineering assistant that *remembers context*, *retrieves domain knowledge*, and *executes tasks* — all safely.



MCP & RAG

Final Thoughts

MCP and RAG aren't just technical components, they represent the future architecture of AI systems.

- RAG = credible knowledge
- MCP = powerful actions
- AI Agents = the intelligence that ties them together

This combination turns AI from a passive responder into a proactive teammate capable of understanding context, retrieving knowledge, and interacting with real systems.

- + AI
- + Mcps
- + Agentic Rag
- + Software Engineering
- + AI Agent



Follow

Published in Dev Genius

32K followers · Last published 6 hours ago

Coding, Tutorials, News, UX, UI and much more related to development



Follow

Written by Vishal Dhok

38 followers · 9 following

No responses yet

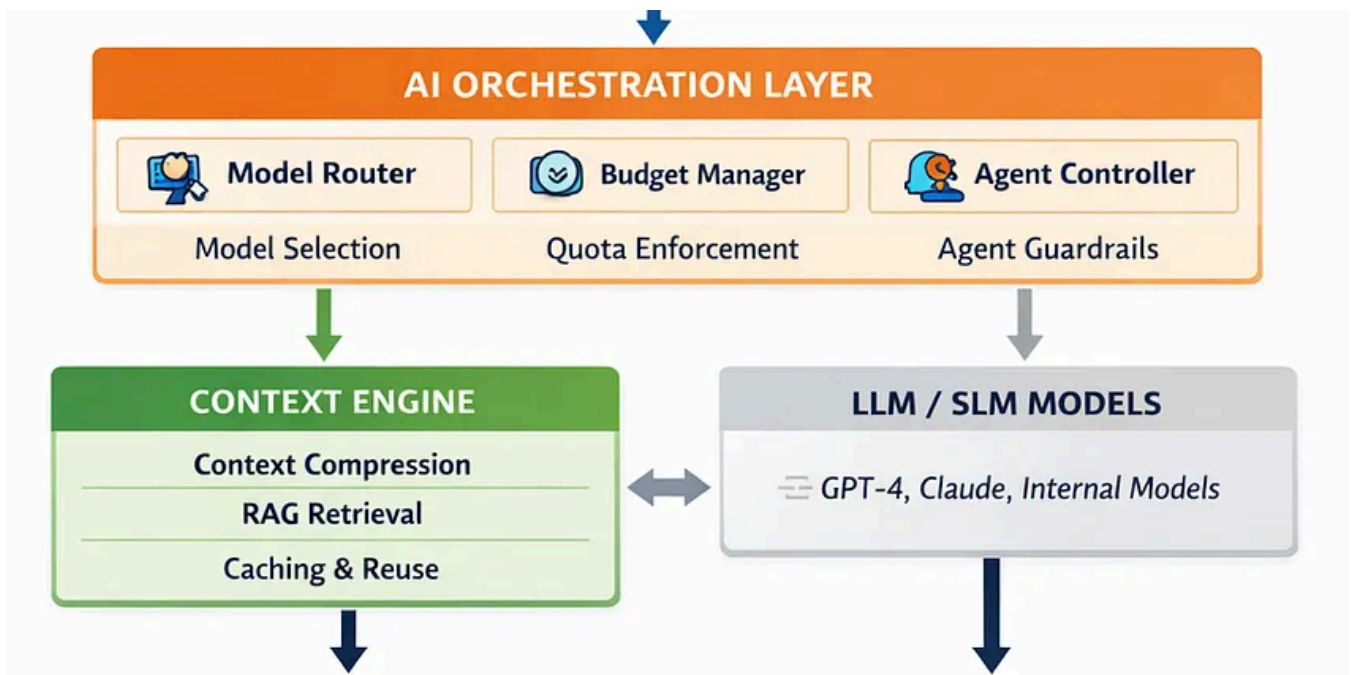


Ramesh Chukka

What are your thoughts?



More from Vishal Dhok and Dev Genius



 In Dev Genius by Vishal Dhok · Mar 7

Building a Cost-Aware AI SDLC Platform

A Reference Architecture for Engineering Leaders

 2

 ...



Most Asked
Java 8
Interview
Coding Questions
(Part-1)



In Dev Genius by Anusha SP · Jan 31, 2023

Java 8 Coding and Programming Interview Questions and Answers

It has been 8 years since Java 8 was released. I have already shared the Java 8 Interview Questions and Answers and also Java 8 Stream API...

👍 1.6K 💬 24 ↻ 1



4. SIDE EFFECTS

LaunchedEffect(key)

```
LaunchedEffect(Unit) {  
    // Runs once when composable  
    // enters composition  
}
```

- Runs a suspend block in a coroutine.
- Restarts when key changes.

SideEffect

```
SideEffect {  
    // Runs after every successful  
    // recomposition  
}
```

- Runs after every successful recomposition.
- For non-suspending side effects only.

5. DERIVED STATE

```
val showButton by remember {  
    derivedStateOf {  
        listState.firstVisibleItemIndex > 0  
    }  
}
```

- Produces a state derived from other state.
- Recalculates only when dependencies change.
- Great for expensive calculations or scroll-based UI.

6. STABILITY

@Stable

Object can change over time but Compose can observe those changes.

Use for classes with mutable properties backed by observable state (e.g. mutableStateOf).

@Immutable

Object never changes after it is created. All properties are immutable.

Compose can skip recomposition if equal.

Remember

Stability helps Compose optimize recomposition by skipping when possible.

7. LISTS & PERFORMANCE

LazyColumn vs Column

LazyColumn	Column
Composes only visible items (lazy).	Composes all items at once.
Memory efficient.	Not suitable for large lists.
Use for long lists.	

8. FLOW, STATE & LIFECYCLE

collectAsStateWithLifecycle

```
val uiState by viewModel.uiState  
    .collectAsStateWithLifecycle()
```

- Collects Flow as State in a lifecycle-aware way.
- Stops collecting when UI is not active.

9. NAVIGATION (BASICS)

```
composable("/home") { HomeScreen() }  
composable("/details/{id}",  
    arguments = listOf(  
        navArgument("id") { type = NavType.StringType }  
    )  
) { backStackEntry ->
```



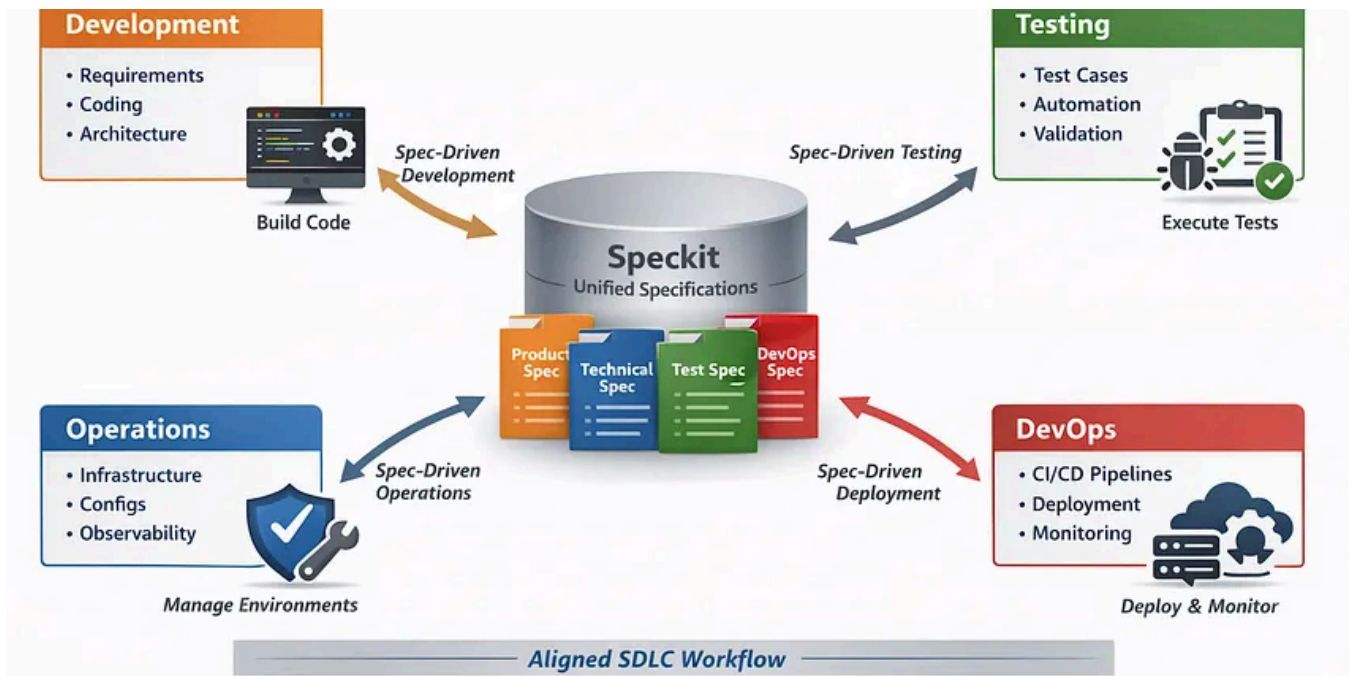
In Dev Genius by Neeraja Gandla · May 29

Jetpack Compose Interview Guide for Experienced Android Developers

Jetpack Compose interviews are interesting because they usually start simple and gradually move deeper into how Compose actually works...

🌟 👍 125 💬 1 ↻ 4





 In Dev Genius by Vishal Dhok · Mar 16

Speckit Template Pack for Spec-Driven SDLC

A Speckit is the single source of truth for a feature, enabling development, testing, and operations to work from the same structured...



See all from Vishal Dhok

See all from Dev Genius

